Une couche de recherche autonome au-dessus de LMLExplore

Proposition de fonctionnement (document de travail)

Adrien Prevost
rédigé avec l'aide de Claude (Anthropic)

13 novembre 2026

Note du 14 novembre 2026 : version 3

Ce document décrit la version 1. La version 3 d'Incognita remplace les *questions* par des *papiers* : feuilles de route approuvées par le propriétaire (avec un fichier `KNOWN.md`), courts papiers écrits par des agents et arbitrés par deux agents (portée et bibliographie, puis preuves), formalisation des papiers acceptés selon la méthode `formalize-paper` (énoncés gelés, puis preuves). Les règles en vigueur sont dans `AGENTS.md` et `CONTRIBUTING.md` ; le reste de ce document n'a pas été mis à jour.

Résumé

LMLExplore fait formaliser par des agents des résultats *connus* de théorie de l'apprentissage, choisis par des humains dans des feuilles de route. Ce document propose une couche au-dessus (Incognita), dans laquelle les agents produisent des résultats *nouveaux*, avec des exigences de qualité au moins égales à celles de LMLExplore, quitte à ralentir. L'idée centrale : Lean règle la question de la vérité d'une preuve, mais pas celles du *sens* des énoncés et de leur *intérêt*, que LMLExplore reçoit aujourd'hui de la littérature et des feuilles de route. Il faut donc reconstruire ces garanties : gel des énoncés, réfutations de premier rang, double formalisation, contrôles des définitions, consolidation périodique de la bibliothèque et niveaux de confiance qui montent avec des vérifications indépendantes et avec l'usage.

Table des matières

1 Point de départ : ce que fait LMLExplore aujourd'hui **2**

2 Objectif et périmètre **3**

3 Rôles, espaces et cycle de vie **3**

3.1 Rôles 3

3.2 Deux espaces 4

3.3 Cycle de vie d'une question 4

4 Validation des questions : humains et agents **4**

5 Règles de production des résultats **5**

6 Consolidation de la bibliothèque (jardinage) 6

6.1 Deux situations différentes 6

6.2 Fonctionnement 7

7 Fiabilité dans le temps 8

7.1 Pourquoi pas des revues à intervalles croissants 8

7.2 Ce que je propose à la place 8

8 Ce qui est repris tel quel de LMLExplore 9

9 Questions ouvertes 9

10 Premiers pas possibles 10

1 Point de départ : ce que fait LMLExplore aujourd'hui

D'après les consignes publiques des dépôts (`AGENTS.md`, `CONTRIBUTING.md` de LMLExplore et de LMLRoadmaps, grilles de revue de Tau Ceti), lues le 13/11/2026 :

— **Les humains écrivent les feuilles de route** (dépôt LMLRoadmaps). Une feuille de route est une spécification précise : un `README.md` qui fait foi (conventions fixées, couches, théorèmes cités par numéro, « chaque élément sans ambiguïté », « rien d'optionnel », « pas de trou ») et un `Suggested.lean` qui prototype définitions et énoncés avec `sorry`. Leur revue juge d'abord : « *Is this the mathematics we want, at the right generality ?* »

— **Les agents écrivent le code**, une PR par sujet, uniquement si elle fait avancer une cible d'une feuille de route. Ils n'ont pas le droit d'ouvrir de PR ni d'issue dans LMLRoadmaps.

— **La CI** exige : pas de `sorry`, axiomes standard seulement, linters de Mathlib.

— **Des relecteurs IA** jugent chaque PR selon des grilles fixes (correctness, reuse, scope, generality, naming, proof quality…). La grille « correctness » est adversariale : chercher un énoncé faux, vide, ou plus faible qu'il ne le prétend.

— **La fusion** est automatique quand toutes les grilles approuvent et que la CI est verte.

— **Consolidation existante** : améliorer, simplifier ou généraliser modestement un lemme est toujours autorisé ; un script détecte les doublons avec les dépendances, qui sont ensuite supprimés.

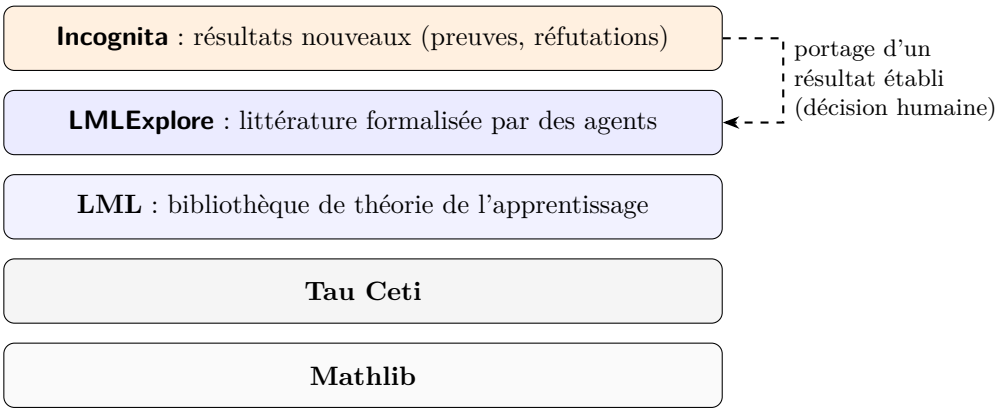
L'hypothèse implicite

Tout ce système suppose que **la vérité et l'intérêt sont acquis en amont** : l'énoncé vient d'un livre ou d'un papier, et un humain a jugé qu'il valait la peine. Les agents n'ont « plus qu'à » traduire fidèlement et prouver, et la revue vérifie la fidélité et la propreté du code.
Conséquence : rien n'est prévu si un énoncé de feuille de route est faux. La PR ne peut pas passer (pas de `sorry`) ; une variante avec hypothèse supplémentaire risque d'être blo-

quée comme « plus faible qu'elle ne le prétend » ; une réfutation n'avance aucune cible ; et l'information ne remonte pas vers la feuille de route, puisque les agents ne peuvent pas y écrire.

2 Objectif et périmètre

Incognita est une bibliothèque Lean qui **dépend** de LMLExplore (donc de LML, Tau Ceti, Mathlib) sans les modifier, à l'image des projets de papier construits sur `lml-paper-template`. C'est un projet **indépendant** : il n'est ni affilié à LMLExplore, LML ou Tau Ceti, ni approuvé par eux, et il ne fait que les utiliser comme dépendances Lean.



Trois choses que LMLExplore reçoit gratuitement doivent être reconstruites :

1. **la vérité** de l'énoncé : on ne la connaît pas à l'avance (Lean la tranche, mais l'énoncé peut s'avérer faux) ;
2. **le sens** des définitions : pour un objet nouveau, il n'y a pas de livre auquel comparer ;
3. **l'intérêt** du résultat : personne n'a encore jugé qu'il valait la peine.

Principe directeur : **ralentir plutôt que dégrader**. Tout le standard de code de LMLExplore est repris tel quel (section 8) ; les règles nouvelles s'y ajoutent.

3 Rôles, espaces et cycle de vie

3.1 Rôles

Rôle	Fait
Humains	approuvent les *programmes* de recherche ; auditent par échantillon ; tranchent les définitions reconnues fausses et les portages vers LMLExplore.
Agents proposeurs	formulent des questions dans un programme approuvé.
Agents chercheurs	explorent, prouvent, réfutent, proposent des variantes.
Agents relecteurs	valident les questions, revoient les PR selon les grilles, font les vérifications indépendantes (section 7).
Agents jardiniers	consolident la bibliothèque (section 6).
Agent bibliographe	indépendant de l'auteur : vérifie les références, lit les passages cités, audite l'attribution de chaque résultat fusionné (section 5).

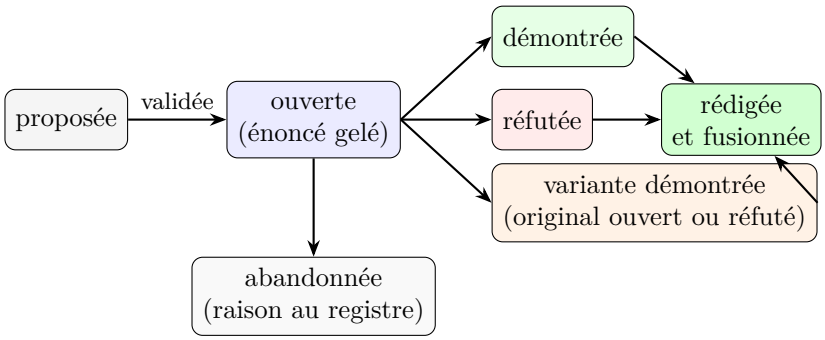
3.2 Deux espaces

— **Exploration** : brouillons, `sorry` permis, jamais fusionnée dans la bibliothèque. C'est là que vit la recherche au sens propre (essais, impasses, contre-exemples numériques).

— **Bibliothèque** : règles de LMLExplore inchangées. Un résultat n'y entre qu'une fois nettoyé.

LMLExplore n'a pas d'espace d'exploration ; un travail de recherche en a besoin, sinon il est poussé à n'essayer que ce qui passera la CI du premier coup.

3.3 Cycle de vie d'une question



Une question réfutée ou résolue par une variante n'est **pas un échec** : c'est souvent le résultat le plus informatif.

Exemple : bandits par lots

La question « B-IMED est-il optimal quand $x_\ell \to x > 1$? » a reçu la réponse « non : il n'est consistant avec aucun poids borné », et c'est un autre algorithme (PT, puis l'hybride HY) qui atteint la constante $x^2/K_{\inf}$. Un système rigide « énoncé → preuve » aurait classé cela comme un échec, alors que la réfutation et la variante étaient les résultats.

4 Validation des questions : humains et agents

Peut-on confier entièrement à des agents la validation des questions ? **En partie.**

Ce qui s'automatise bien :

— la question est-elle bien posée (objets définis, quantificateurs clairs) ;

— est-elle déjà résolue dans la littérature (recherche bibliographique outillée) ;

— les hypothèses sont-elles satisfiables simultanément ;

— les définitions retombent-elles sur les cas connus ;

— est-elle triviale à partir de la bibliothèque (un agent tente une preuve rapide).

Ce qui résiste : l'intérêt. Le problème n'est pas qu'un agent juge mal un cas isolé, mais que ses erreurs sont *corrélées* avec celles des agents chercheurs (mêmes modèles, mêmes biais). Une boucle entièrement automatique tend à dériver vers ce que les IA trouvent naturel et faisable, plutôt que vers ce qui compte pour le domaine, et peut produire une branche entière cohérente, vérifiée et sans intérêt sans qu'aucun mécanisme interne ne le détecte.

Règle – les humains valident des programmes, les agents des questions

1. Un humain approuve un **programme de recherche** (par exemple « bandits par lots : constante optimale en fonction de la grille »), avec un texte court : motivation, objets, ce qui serait un résultat important. Coût : quelques minutes par mois.
2. À l'intérieur d'un programme, un **agent relecteur valide les questions** selon les critères automatisables ci-dessus.
3. Un humain fait un **audit par échantillon** : il lit de temps en temps quelques questions acceptées tirées au hasard. S'il en trouve de mauvaises, on resserre les critères.

5 Règles de production des résultats

Règle – gel de l'énoncé (anti-Goodhart)

Un agent qui n'arrive pas à prouver un énoncé a tout intérêt à l'affaiblir jusqu'à ce qu'il passe. Donc :

— l'énoncé cible est fixé dans un fichier séparé **avant** tout travail de preuve ; `comparator` vérifie que la preuve porte exactement sur lui ;

— une **variante** n'est acceptée qu'avec une justification formalisée : contre-exemple à l'original, ou preuve que l'hypothèse ajoutée est nécessaire, ou étiquette explicite « original toujours ouvert » ;

— un affaiblissement silencieux bloque la PR.

Règle – les réfutations sont des résultats

Un contre-exemple formalisé (preuve de la négation de l'énoncé gelé) est fusionné comme une preuve, avec la même exigence. Il ferme la question et alimente le registre.

Règle – double formalisation des énoncés principaux

Pour un théorème principal ou une définition nouvelle importante, deux agents indépendants formalisent l'énoncé à partir du même texte mathématique, sans voir le travail l'un de l'autre. On prouve en Lean que les deux versions sont équivalentes. C'est la meilleure défense contre une formalisation fausse quand il n'existe pas de livre auquel comparer.

Règle – contrôles de toute définition nouvelle

En plus du témoin non trivial déjà exigé par LMLExplore :

— **retour aux cas connus** : sur un cas particulier, l'objet coïncide avec la notion classique et le théorème redonne le résultat publié ;

— **non-vacuité conjointe** : une instance concrète satisfait *toutes* les hypothèses du théorème principal en même temps ;

— **finesse**, quand c'est possible : un exemple où retirer une hypothèse rend le résultat faux.

Pourquoi insister sur les définitions

Un théorème faux est arrêté par le noyau. Une définition subtilement mal choisie passe partout et rend tout ce qui suit vrai mais hors sujet. C'est là que se cache le risque principal d'une bibliothèque de résultats nouveaux.

Règle – relecteurs indépendants

Les relecteurs tournent sur des modèles ou des consignes différents de ceux de l'auteur et ne voient pas son raisonnement : ils reçoivent l'énoncé mathématique et l'énoncé Lean, doivent les reformuler eux-mêmes et chercher un contre-exemple.

Règle – registre des impasses

Chaque question a un fichier qui recense les approches essayées et la raison de leur échec (« tentative X, échoue parce que Y »), consultable par les agents suivants. Aujourd'hui une tentative ratée disparaît avec la PR fermée ; en recherche, savoir ce qui ne marche pas vaut presque un théorème.

Règle – nouveauté et attribution

Un résultat n'est jamais déclaré nouveau par son auteur : il entre au catalogue avec la nouveauté « non vérifiée », et seul l'audit d'un agent **bibliographe**, indépendant de l'auteur, peut la passer à « nouveau », « redécouverte de… » ou « connu ». L'audit crédite aussi les **techniques**, pas seulement les résultats : d'où vient chaque idée de la preuve. Un énoncé nouveau peut reposer sur une technique ancienne, comme la technique de changement de mesure par lots qui remonte à Lorden (1983).
Les modèles inventent facilement des références plausibles, et Lean ne les voit pas. Une citation n'est donc permise que vers une entrée de la bibliographie commune (`literature/`) dont les métadonnées ont été vérifiées et dont le passage cité est marqué comme lu, avec qui l'a lu et quand. Chaque rédaction contient une rubrique « Attribution ».

Règle – le texte est un livrable

Un résultat n'est terminé que lorsqu'il existe une preuve rédigée et lisible par un humain, avec la correspondance énoncé par énoncé vers le Lean. Sinon on obtient des théorèmes vrais que personne ne comprend, donc que personne ne réutilise.

6 Consolidation de la bibliothèque (jardinage)

Avec des agents qui produisent vite, la bibliothèque devient une pile de résultats intermédiaires si personne ne la réorganise. C'est ce que font d'habitude les synthèses et les livres. Je propose d'en faire une tâche de premier rang, avec son type de PR et sa grille de revue.

6.1 Deux situations différentes

Englobement. Un théorème général B implique un cas particulier A.

Exemple : bandits par lots

Un algorithme pour $x = 1$, un autre pour $x > 1$, puis l'hybride HY pour tout $x \geq 1$. Mais HY a des constantes astronomiques, alors que B-IMED à $x = 1$ a des constantes raisonnables : supprimer B-IMED comme « englobé » ferait perdre le seul algorithme utilisable en pratique.

Règle – englobement quantitatif

A n'est déclaré englobé par B que si B implique A **avec un contenu quantitatif au moins aussi bon** (mêmes constantes, même ordre, mêmes hypothèses), vérifié en Lean. La preuve de A devient alors « corollaire de B », mais l'énoncé A reste. Sinon les deux restent, et la documentation dit ce que chacun apporte.

Changement de référence. On a prouvé des algorithmes « optimaux » au sens de Lai–Robbins, puis une borne inférieure plus fine apparaît. Les bornes supérieures prouvées restent vraies ; c'est le *mot* « optimal » qui devient trompeur.

Règle – pas de mot évaluatif dans une définition

Pas de `def IsOptimal` défini comme « atteint Lai–Robbins ». On énonce séparément la borne supérieure (constante explicite) et la borne inférieure (classe d'algorithmes explicite) ; « optimal » est un **corollaire nommé** qui associe les deux dans une classe donnée. Une borne plus fine ajoute une nouvelle association ; l'ancienne reste vraie et reçoit le statut « supplantée par… ».

Un détecteur d'erreurs gratuit

Si une borne inférieure est strictement plus grande qu'une borne supérieure prouvée dans la même classe, Lean en déduit `False` : c'est le signe qu'une définition est fausse quelque part. Une tâche automatique qui tente de combiner bornes inférieures et supérieures de même classe détecte ainsi les erreurs de définition.

6.2 Fonctionnement

1. **Catalogue des résultats** : pour chaque théorème principal, des métadonnées (quantité bornée, classe de problèmes et d'algorithmes, régime asymptotique ou non, constantes, statut). C'est ce qui permet de repérer les candidats ; sans catalogue, un agent ne voit que des noms de théorèmes.

2. **Tâche périodique de jardinage** : tenter les implications entre résultats en Lean (peu coûteux), repérer les familles de cas particuliers qui appellent une unification, tenir à jour l'*état de l'art* de chaque question (meilleur résultat, historique).

3. **Actions classées par risque** :

Action	Qui décide
Refaire la preuve de A comme corollaire de B	agents et relecteurs IA
Changer un statut (« supplanté », « englobé »)	relecteurs IA, justification au catalogue
Supprimer un énoncé	jamais automatiquement, sauf vrai doublon
Proposer une unification (comme HY)	c'est une *question de recherche* : validée comme les autres

Règle – aucun théorème perdu

Toute PR de consolidation qui retire ou modifie un théorème doit montrer, vérifié par `comparator`, que l'ancien énoncé reste prouvable à partir de la bibliothèque. Seule exception : une définition reconnue fausse, et c'est un humain qui tranche. Le jardinage est l'endroit le plus risqué : un agent peut « simplifier » en supprimant de la connaissance.

7 Fiabilité dans le temps

La confiance dans un résultat doit monter avec le temps : un résultat tout juste fusionné ne vaut pas un résultat utilisé depuis six mois. La question est *ce qui* doit la faire monter.

7.1 Pourquoi pas des revues à intervalles croissants

Une idée naturelle est la répétition espacée : revoir un résultat à 1 jour, puis 3 jours, etc. Elle est moins efficace qu'il n'y paraît :

— **une preuve Lean ne s'oublie pas** : le noyau donne la même réponse le premier et le centième jour ;

— **revoir l'énoncé avec le même type de relecteur et les mêmes informations donne des vérifications corrélées** : un relecteur qui a laissé passer une définition subtilement fausse la laissera probablement passer à nouveau. Cinq revues identiques ne valent pas cinq vérifications indépendantes, et donnent un faux sentiment de sécurité ;

— **ce qui change avec le temps, c'est le contexte** (résultats voisins, usages, définitions nouvelles), et c'est lui qui fait apparaître les erreurs.

7.2 Ce que je propose à la place

Règle – revues déclenchées par des événements

On relit un résultat quand un nouveau théorème l'utilise, quand un résultat voisin est fusionné, quand un corollaire paraît suspect, quand on en tire une contradiction, ou quand il est englobé lors du jardinage.

Règle – diversité plutôt que répétition

Chaque revue supplémentaire est d'un **type différent** : recherche de contre-exemples ; retour aux cas connus ; **rétrotraduction** (un agent qui ne voit que le Lean réécrit l'énoncé en mathématiques, puis on compare au texte d'origine) ; double formalisation ; si possible, un autre modèle.

Règle – niveaux de confiance affichés

Niveau	Condition
fusionné	CI verte et grilles approuvées (standard LMLExplore)
contrôlé	vérifications indépendantes de types différents réussies (au moins trois)
utilisé	consommé par des théorèmes en aval, contrôles de cohérence passés
établi	rédigé, relu par un humain, aucune alerte depuis une génération de relecteurs

Les seuils sont à calibrer ; l'essentiel est que le niveau monte avec *l'information nouvelle* apportée, pas avec le temps écoulé.

Règle – réaudit à chaque génération de relecteurs

La seule revue périodique conservée : quand un modèle relecteur nettement meilleur sort, toute la bibliothèque est revue. Là, la revue apporte une vraie information, parce que le nouveau relecteur est moins corrélé avec les anciens.

8 Ce qui est repris tel quel de LMLExplore

— CI verte, pas de `sorry`, axiomes `propext`, `Classical.choice`, `Quot.sound` seulement, linters de Mathlib, pas de `set_option`.

— Un sujet par PR, pas de PR empilées, réutilisation obligatoire de l'existant (Mathlib, Tau Ceti, LML, LMLExplore), pas de copie de ce qui existe déjà.

— Grilles de revue de Tau Ceti (correctness, reuse, scope, generality, naming, proof quality…), auxquelles s'ajoutent des grilles propres : gel de l'énoncé, nouveauté, contrôles des définitions, consolidation.

— Fichiers d'infrastructure possédés par les humains.

— Jamais de contribution automatique vers les dépendances : le portage d'un résultat établi vers LMLExplore se fait sur décision humaine, comme pour les projets de papier.

9 Questions ouvertes

Point ouvert

— **Crédit et paternité** : qui est l'auteur d'un résultat trouvé par un worker ? Comment le citer ? Quel rapport avec une publication en revue ?

— **Coût** : double formalisation, revues diverses et jardinage multiplient le calcul par résultat. C'est voulu (ralentir plutôt que dégrader), mais à chiffrer.

— **Diversité des modèles** : l'indépendance des relecteurs suppose d'avoir accès à des modèles réellement différents.

— **Outillage du catalogue** : métadonnées en Lean (attributs), en TOML à côté, ou extraites automatiquement ?

— **Calibrage** : seuils des niveaux de confiance, fréquence des audits humains, critère d'abandon d'une question.

— **Conditions d'utilisation** des fournisseurs de modèles pour un fonctionnement autonome prolongé.

10 Premiers pas possibles

1. Un dépôt qui dépend de LMLExplore à version épinglée, avec un espace d'exploration et un espace bibliothèque : c'est `https://github.com/prevostad/Incognita` (règles des agents dans `AGENTS.md`).
2. Un seul programme pilote, sur un domaine où l'on dispose déjà de résultats vérifiés pour calibrer : par exemple les bandits par lots (cas $x = 1$, $x > 1$, hybride HY).
3. Les règles les moins coûteuses d'abord : gel de l'énoncé, réfutations acceptées, registre des impasses, catalogue minimal.
4. Ensuite la double formalisation et les niveaux de confiance, une fois mesuré ce qui attrape réellement des erreurs.